

NCTU-EE ICLAB – Autumn 2024

Lab11 Exercise:

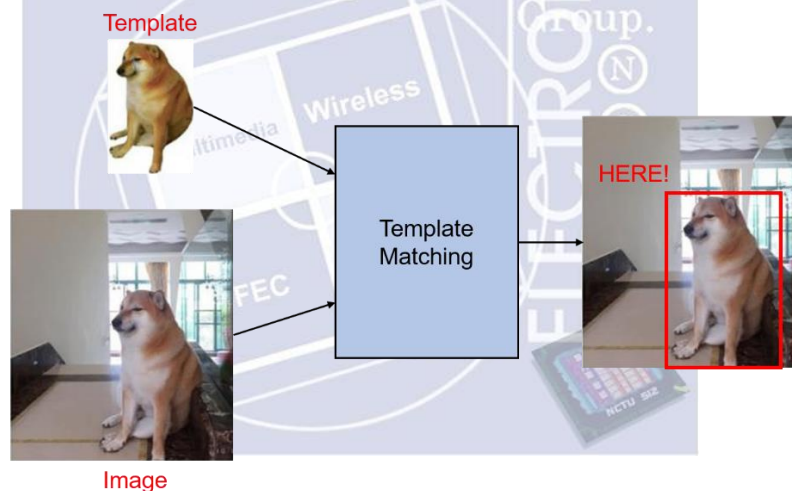
Design: Template Matching with Image Processing (TMIP)

Data Preparation

- Extract the test data from TA's directory:
`% tar xvf ~iclabTA01/Lab11_2024A.tar`
- The extracted LAB directory contains:
 - Practice/
 - Exercise/

Design Description

Template Matching can be seen as a very basic form of object detection. It is a high-level machine vision technique that identifies the parts on an image that match a predefined template, which means we can detect objects in an input image using a “template” containing the object we want to detect. For example,



There are lots of methods of calculating image similarity for finding the matching parts, such as “Sum of Square Differences”, “Cross Correlation”, “Cross Coefficient”, etc. In this exercise, you are asked to use the “Cross Correlation” method, which is a dot product by taking every pair of pixels and multiply, then sum all products. Its formula is shown as below.

$$R(x, y) = \sum_{x', y'} (\text{Template}(x', y') \times \text{Image}(x + x', y + y'))$$

Image processing is a method to convert an image into digital form and perform some operations on it, in order to get an enhanced image or to extract some useful information from it.

In this exercise, you need to build a design to perform template matching by using correlation method and doing some image processing. For the matrix requires large space to store, you are asked to use memory for finishing this lab.

- Image & Template

- Size (Row × Col)

The **image size** will be 4×4, 8×8 and 16×16 according to the given size value which will be given at the beginning of each pattern. The **template size** will only be 3×3.

- Element

The elements of an **image** are represented in 24-bit RGB format. The elements of a **template** are 8-bit unsigned integer.

In order to reduce the number of IO ports, the R, G and B value of one element will be given sequentially in 3 cycles.

- Action

There are 8 possible operations will be given:

- Grayscale Transformation (Maximum method)

Find the maximum of the RGB components directly. We will use the function shown as below.

$$\text{grayscale}(x,y) = \max\{\text{red}(x,y), \text{green}(x,y), \text{blue}(x,y)\}$$

- Grayscale Transformation (Average method)

Calculate the average of the RGB components directly. We will use the function shown as below. In this operation, the result might be in float format, so we will transform it into integer with round-down approach.

$$\text{grayscale}(x,y) = \frac{\{\text{red}(x,y) + \text{green}(x,y) + \text{blue}(x,y)\}}{3}$$

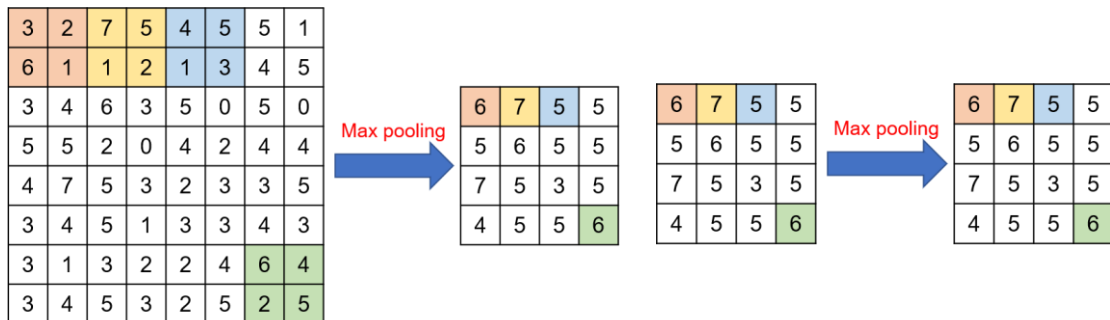
- Grayscale Transformation (Weighted method)

Calculate the weighted average of the RGB components based on human eye sensitivity. We will use the function shown as below. In this operation, whenever division operations occur, simply transform the result into integer with round-down approach.

$$\text{grayscale}(x,y) = \frac{\text{red}(x,y)}{4} + \frac{\text{green}(x,y)}{2} + \frac{\text{blue}(x,y)}{4}$$

➤ Max Pooling

The 2×2 filter will slide through the data matrix, and the maximum pixels will be picked out and the rest will be discarded. The stride size will be 2 in max pooling operation. This operation will be performed only when the image size is 8×8 or 16×16 . If the image size is 4×4 , the result of max pooling will be the same as the original matrix.



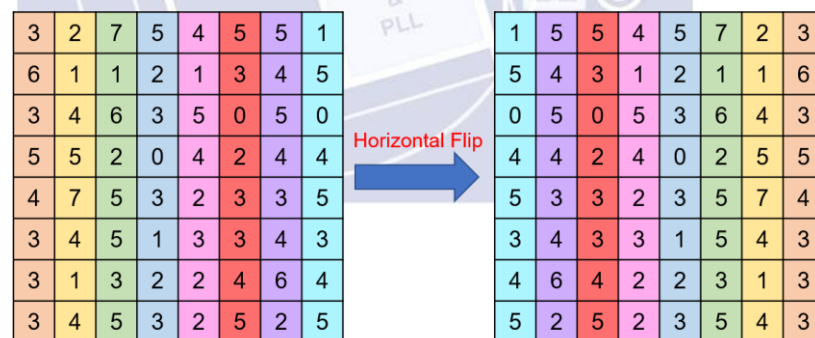
➤ Negative

In a negative film, light areas appear dark and dark areas appear light, which is the opposite of the image as seen in the real world. We will use the function shown as below.

$$\text{negative}(x, y) = 255 - \text{grayscale}(x, y)$$

➤ Horizontal Flip

Mirror the image in the horizontal direction, which means each pixel will exchange its value with the corresponding horizontal column.

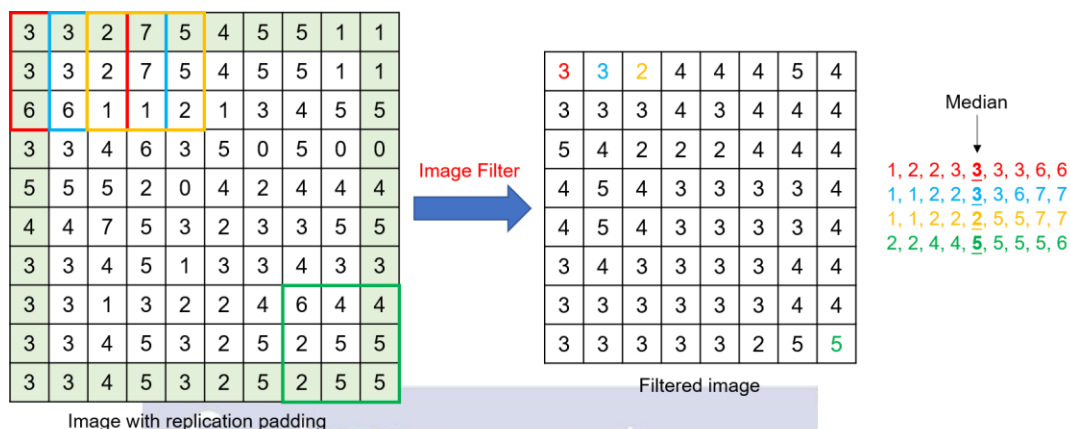


➤ Image Filter

We usually do noise cancelling or enhance the edges of specific objects on images with image filter. In this operation, we will use median filter with 3×3 mask to implement this function. The steps for medium filter are shown as below.

- Step 1: Apply replication padding on the image.
- Step 2: Collect the element values within the 3×3 window.

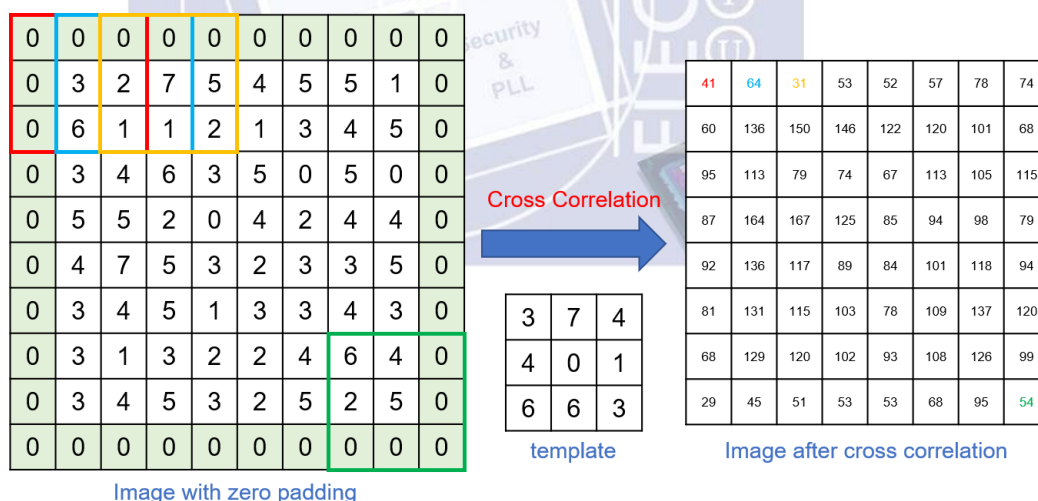
- Step 3: Find the **median value** in the list of collected element values.
- Step 4: Replace the original center element value with the median value.
- Step 5: Repeat step 2~5 to slide through the image with **stride of 1**.



➤ Cross correlation

This operation is the main goal of our design to do template matching. The steps are shown as below.

- Step 1: Apply **zero padding** on the image.
- Step 2: Do **convolution (stride of 1)** on the image with the 3×3 template.



● Rules of input data

In this exercise, you will get the image, image size and template at the beginning for each pattern. After that, you will get several numbers which range in 0 to 7 as a set of actions. The following are some rules about the actions:

- The number of actions ranges from **2 to 8** for each set.
- The first action is the **grayscale transformation**, which represents as 3'b000, 3'b001 or 3'b010.
- The last action is the **cross correlation**, which represents as 3'b111.
- **Only one of the actions 0, 1, or 2 will appear once** and it must be the **first action** in each set.
- The **action 7 will only appear once** and must be the **last action** in each set

After 8 sets of action, you will get the next pattern.

- Rules of output result

When finishing a set of actions, you need to output the result matrix in **raster scan order** with **serial out format from the MSB**.

- The flow for input/output

Flow	Pattern 1	Set1		Set2		...	Set 8		Pattern 2	Set1		...
Behavior	Input	Input	Output	Input	Output		Input	Output	Input	Input	Output	
Objects	image template image_size	action	result matrix (serial out)	action	result matrix (serial out)		action	result matrix (serial out)	image template image_size	action	result matrix (serial out)	

First action	0~6 actions	Last action
0~2	3~6	7

- Example

At the beginning, you will obtain the image consisting of 8-bit values for red, green and blue channel, image size and template.

image_size = 1 (8×8)

6	2	5	7	7	6	1	1
4	2	0	4	1	3	4	1
0	8	7	6	8	1	9	1
9	9	2	2	6	0	0	7
2	9	0	6	0	7	9	0
7	2	0	0	2	3	3	5
4	0	5	9	4	2	1	7
4	4	8	2	1	6	5	4

Red channel

0	5	8	4	5	9	8	1
8	2	3	1	0	4	3	9
5	3	7	2	2	0	6	0
4	6	0	0	5	5	6	5
8	9	9	1	2	5	1	7
0	4	9	1	7	6	8	2
3	3	4	1	0	9	9	3
3	3	3	5	3	8	1	9

Green channel

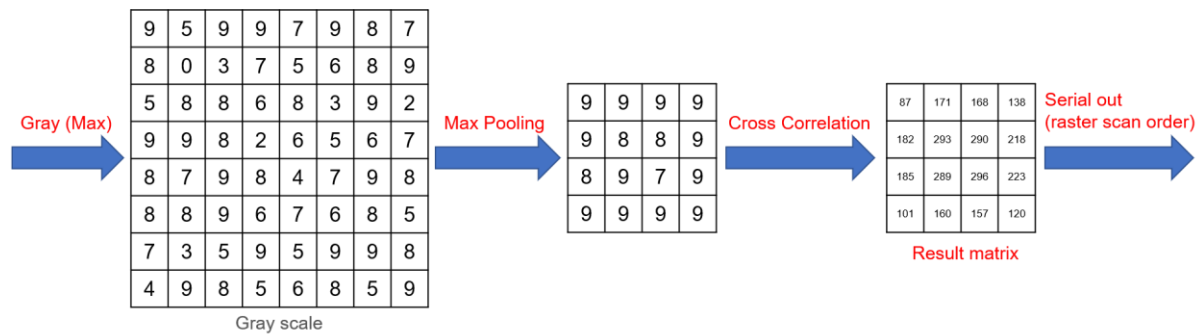
9	2	9	9	4	0	6	7
4	0	2	7	5	6	8	5
5	6	8	4	8	3	1	2
6	1	8	2	6	1	6	5
1	7	5	8	4	3	4	8
8	8	7	6	0	0	1	5
7	1	3	3	5	1	9	8
4	9	8	4	6	2	5	0

Blue channel

3	7	4
4	0	1
6	6	3

Template

Then you will get the action numbers in sequence. In this case, the sequence of the action number is 0 → 3 → 7, which means doing “Grayscale conversion (Maximum method)”, “Max pooling” and “Cross Correlation”.



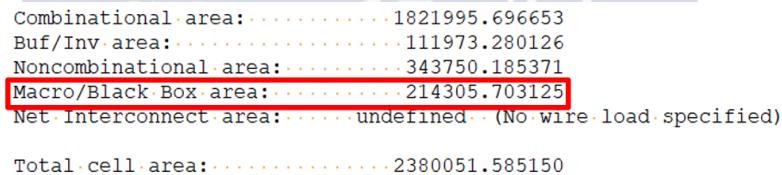
I/O Description

Input	Bit Width	Definition and Description
clk	1	Clock
rst_n	1	Asynchronous active-low reset
in_valid	1	High when image , template and image_size are valid.
in_valid2	1	High when action is valid.
image	8	Element of input image . It will be sent in raster scan order continuously when in_valid is high. The RGB data for each element in the matrix will be provided sequentially over three cycles from the 8-bit image. The RGB data for the next element will be provided in the subsequent three cycles, continuing this process until all elements have been delivered. (Example: R ₁ , G ₁ , B ₁ , R ₂ , G ₂ , B ₂ , ..., R _n , G _n , B _n)
template	8	Element of input template . It will be sent in raster scan order continuously when in_valid is high. The elements are unsigned integers.
image_size	2	Size of the input image matrix. It will be given in the first cycle when in_valid is high. 2'd0: 4×4. 2'd1: 8×8. 2'd2: 16×16
action	3	The signals will be given when in_valid2 is high. The definition is as following: 3'd0: Grayscale Transformation (Max method) 3'd1: Grayscale Transformation (Average method) 3'd2: Grayscale Transformation (Weighted method) 3'd3: Max Pooling 3'd4: Negative 3'd5: Horizontal Flip 3'd6: Image Filter 3'd7: Cross Correlation
Output	Bit Width	Definition and Description
out_valid	1	High only when out_value is valid. It cannot be overlapped with in_valid and in_valid2
out_value	1	It will output the final result matrix in raster scan order and serial out format from the MSB after finishing a set of action. Each out_value contains 20 bits for an element of the result matrix.

Specification

Top module

1. Top module name: **TMIP** (file name: **TMIP.v**)
2. You can adjust your clock period by yourself, but the maximum period is **20 ns**. The precision of clock is 0.1, for example, 19.5 is allowed but 19.55 is not.
3. The execution latency is limited to **5000 cycles**. The latency is the clock cycles between the falling edge of the last cycle of **in_valid2** and the rising edge of the first cycle of **out_valid**.
4. The total cell area should not larger than **1,500,000 μm^2** .
5. In this lab, you **must use the memory and generate it yourself**. The number of words and the bits per each word is defined by yourself. The total number and kind of memory is unlimited. We will check it at **TMIP.area** in **02_SYN/Report**. The area of **Macro/Black Box** must not be 0. The example is shown in following figure.



```
Combinational area:.....1821995.696653
Buf/Inv area:.....111973.280126
Noncombinational area:.....343750.185371
Macro/Black Box area:.....214305.703125
Net Interconnect area:.....undefined (No wire load specified)

Total cell area:.....2380051.585150
```

6. If any port of memory is connected with mismatched width, the memory will not be synthesized and you will get an error message. Even though the design may still pass gate level simulation, this situation will be regarded as a synthesis fail. In this case, the memory area will be 0 in **TMIP.area**. We will check it at **syn.log** and **TMIP.area**.

Reset

7. It is **asynchronous reset** and **active-low** architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals that should be reset after the reset signal is asserted.
8. The reset signal (**rst_n**) will be given only once at the beginning of simulation. **All output signals should be reset to 0 after the reset signal is asserted.**

in_valid & input signals

9. All input signals are synchronized at negative edge of the clock.
10. **in_valid** will come after reset.
11. The input of **image** is delivered in **raster scan order** for **3×current size of image (4×4, 8×8, 16×16) cycles continuously** when **in_valid** is tied high. **When all elements of an image are delivered completely, it is tied to unknown state, and in_valid will also be tied low.**

12. The input of **template** is delivered in **raster scan order** for **9 cycles continuously** when **in_valid** is tied high. **When all elements of a template are delivered completely, it will be tied to unknown state.**
13. **image_size** will be given in **the first cycle** when the **in_valid** is high.
14. Every time **in_valid2** is triggered, it is tied high for several cycles depending on **the numbers of actions continuously**. When **in_valid2** is low, the input of **action** is tied to unknown state.
15. **in_valid2** will be triggered for **a total 8 times** after **in_valid** falls in a single pattern. After each time **in_valid2** triggers, your design will do template matching with image processing and then **out_valid** will be tied high for corresponding cycles.
16. In each pattern, the first **in_valid2** will come in **2~4 negative edge of clock** after **in_valid** falls, and the other 7 times **in_valid2** will come in **2~4 negative edge of clock** after **out_valid** falls.

out_valid & out_value

17. All output signals should be synchronized at clock positive edge.
18. The TA's pattern will capture your output for checking at **clock negative edge**.
19. **out_valid** cannot overlap with **in_valid** and **in_valid2** at any time.
20. **out_valid** is limited to be high only when **out_value** is valid.
21. **out_value** should be correct when **out_valid** is high. And **out_value** should be low when **out_valid** is low.
22. **out_value** must be delivered for several cycles which depends on **current size of matrix continuously**, and **out_valid** should be high simultaneously.
23. Each **out_value** contains **20** bits for an element of the result matrix.
24. The next input pattern will come in **2~4 negative edge of clock** after the eighth **out_valid** of this pattern falls.

Synthesis

25. The input delay is set to **0.5*(clock period)**.
26. The output delay is set to **0.5*(clock period)**, and the output loading is set to **0.05**.
27. The input delay of **clk** and **rst_n** should be zero.
28. The synthesis time should be less than **1 hour**.
29. The synthesis result (syn.log) of data type cannot include any **latches** and **error**.

30. After synthesis, you can check **TMIP.area** and **TMIP.timing**. The area report is valid when the slack in the end of timing report should be **non-negative** and the result should be **MET**.

Gate-level simulation

31. The gate-level simulation cannot include any timing violations without the *notimingcheck* command.

APR

32. Floorplanning

- Core size: Defined by user
- Core to IO boundary: Each side must be more than **100**
- Hard Macro placement: All hard macro should be **in CORE**

33. Powerplanning

- Core Ring
 - (i) Top & Bottom: metal layer must be **odd** (1,3,...) and width is **9**
 - (ii) Left & Right: metal layer must be **even** (2,4,...) and width is **9**
 - (iii) Each side must be **wire group, interleaving**, and at least **5 pairs**.
- Stripes
 - (i) Horizontal: metal layer must be **odd** (1,3,...) and width is at least **2**
 - (ii) Vertical: metal layer must be **even** (2,4,...) and width is at least **2**
 - (iii) The maximum distance between two stripes or stripe and edge should be less than **200**

34. Timing analysis results

- Timing slack: **No negative slacks** after setup/hold time analysis (including SI)
- Design Rule Violation (DRV): The **DRV of (fanout, cap, tran)** should be **all 0** after post-Route setup/hold time analysis (including SI)

35. Design verification results

- Layout v.s. Schematic (LVS): **NO LVS violations** after “verify Connectivity”
- Design Rule Check (DRC): **NO DRC violations** after “verify DRC”

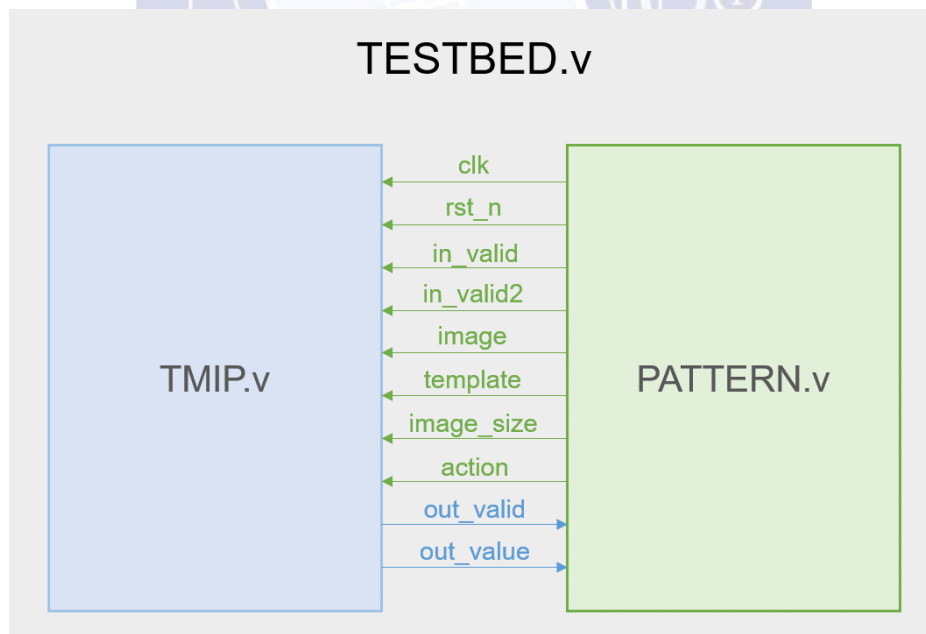
Post-simulation

36. The post-simulation cannot include any timing violations without the *notimingcheck* command.

Supplement

37. In this lab, you are **NOT** allowed to use Designware IP.
38. Don't use any wire/reg/submodule/parameter name called **error**, **congratulation**, **latch** or **fail** otherwise you will fail the lab. Note: *** means any char in front of or behind the word. e.g: error_note is forbidden.
39. Don't write Chinese or other language comments in the file you sent.
40. Verilog command `//synopsys dc_script_begin`, `//synopsys dc_script_end` `//synopsys translate_off`, `//synopsys translate_on` are only allowed during the usage of including and setting designware IPs, other design compiler optimizations are forbidden.
41. Using the above commands are allowed, however any error messages during synthesis and simulation, regardless of the result will lead to failure in this lab.
42. Any form of display or printing information in verilog design is forbidden. You may use this methodology during debugging, but the file you turn in should not contain any coding that is not synthesizable.

Block Diagram



Grading Policy

The performance is determined by the area and latency of your chip. The less cost your design has, the higher grade you get. In this lab, you can use the RTL code provided by TA if you need.

Use your own design

- RTL/Gate-level/Post-level simulation, Synthesis and APR correctness: 70%
- Performance: 30%
 - $\text{Chip area}^2 * \text{Total latency}$
 - $\text{Total latency} = \text{cycle time} * \text{latency}$

Use TA provided design

- APR and Post-level simulation correctness: 60%
- Performance: 10%
 - $\text{Chip area}^2 * \text{Total latency}$
 - $\text{Total latency} = \text{cycle time} * \text{latency}$

Note

- Please submit your design in Lab11/09_SUBMIT
 - 1st_demo deadline : 2024/12/02 (Mon.) 12:00:00
 - 2nd_demo deadline: 2024/12/04 (Wed.) 12:00:00 (score will be 30% off)
- Please upload the following files under 09_SUBMIT
 - RTL design: **TMIP.v** (if you use TA's code, you don't need to submit this file)
 - filelist: **filelist.f**
 - Memory file: **all your memory files .v / .db / .lef / .lib in 04_MEM**
 - APR file: **CHIP.sdc / CHIP.inn / CHIP.io / CHIP.v / CHIP.sdf / CHIP.inn.dat**
 - In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It's means that the TA will demo your design under this clock cycle time.
 - If your files **violate the naming rule**, you will get **10 deduction points**.
- Template folders and reference commands:
 - 01_RTL/ (RTL simulation) **./01_run_vcs_rtl**
 - 02_SYN/ (Synthesis) **./01_run_dc_shell**

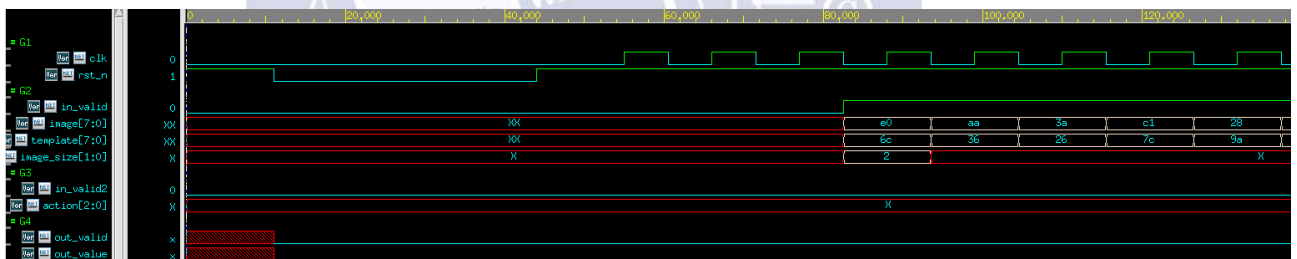
- Check if there is any latch in your design in syn.log
- Check the timing of design in /Report/TMIP.timing
- Check the area of design in /Report/TMIP.area
- 03_GATE/ (Gate-level simulation) **./01_run_vcs_gate**
- 04_MEM/ (Memory location)
- 05_APR/ (APR Folder) **./00_combine**
- 06_POST/ (Post-level simulation) **./01_run_vcs_post**
- You should generate your own memory and put the required files here
- 09_SUBMIT/ (Submit your files) **./00_tar**

./01_submit

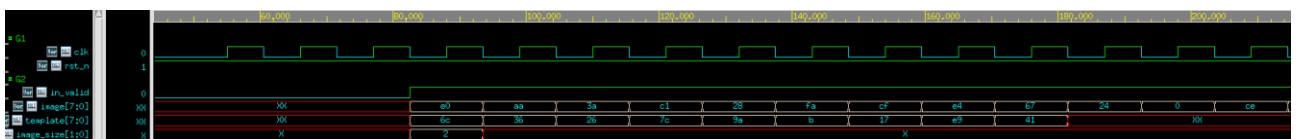
./02_check

Reference Waveform

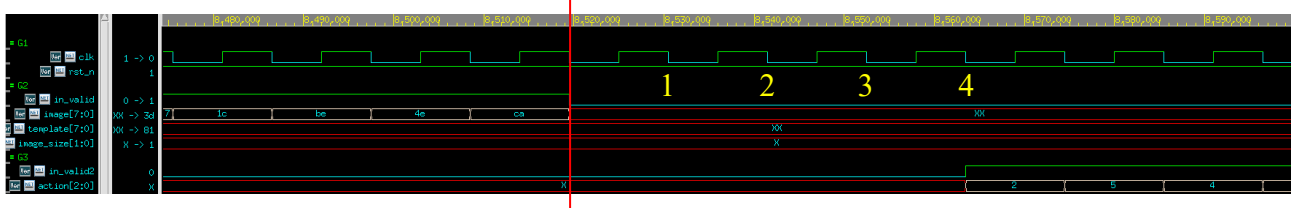
- Asynchronous reset and active-low reset all output to low



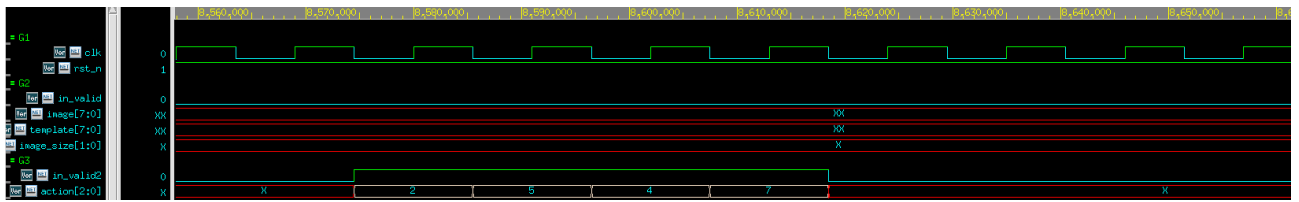
- Input of image, template and image_size



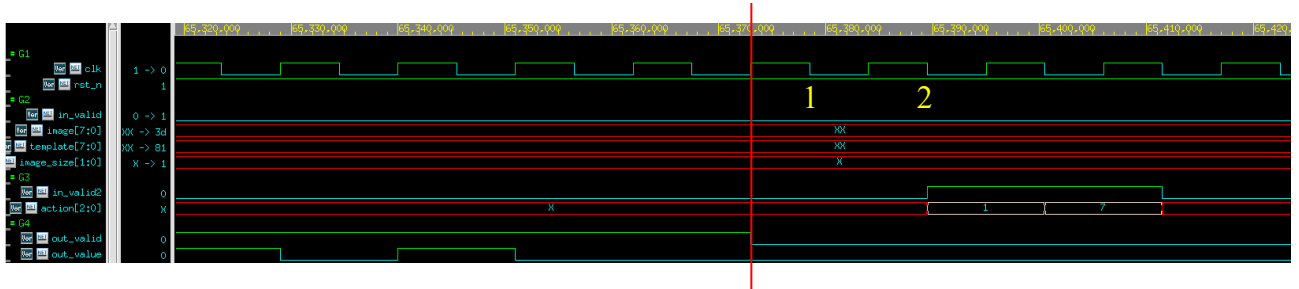
- in_valid2 will come in 2~4 negative edge of clock after in_valid is tied low



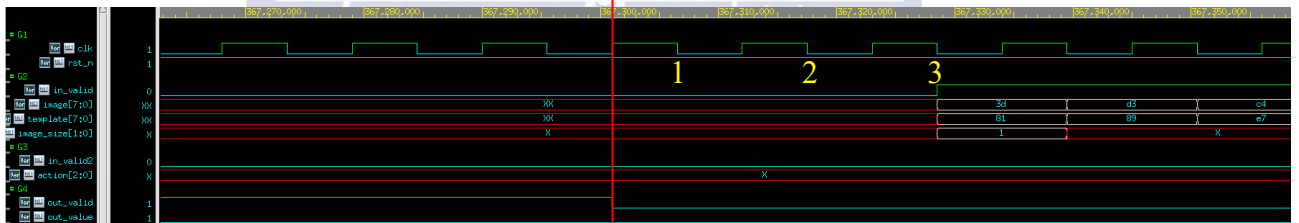
- Input of action (In this case, the number of actions is 4)



- The other 7 times in_valid2 will come in 2~4 negative edge of clock after out_valid falls



- The next input pattern will come in 2~4 negative edge of clock after the eighth out_valid of this pattern falls



- The definition of latency

